

AI for Co-folding — Hackathon Guide

Welcome! This guide contains everything you need to get started: the challenge brief, the scientific context for the target, how to access your team's machine, and how to run fine-tuning jobs in ApherisFold.

Each team is given access to a GPU-enabled machine and a deployment of ApherisFold, which provides a streamlined UI for fine-tuning without needing to touch the underlying stack. Teams who want to go deeper — modifying the model architecture, customising the training loop, or running their own tooling — are also free to use the provisioned machine as bare metal via SSH. Either way, it's the same VM — use whichever tooling fits what you're trying to do.

What You're Building

By the end of the hackathon, each team gives a short presentation on what they built and what they found. Format and content are largely up to you — every team's story will look a little different.

Strong presentations tend to cover some mix of:

- What you set out to do and why — the question or hypothesis behind your approach.
- What you actually changed — dataset, hyperparameters, architecture, training recipe, or tooling.
- Evidence it worked (or didn't) — quantitative results where possible (e.g. pose RMSD on held-out structures, training/validation curves), qualitative observations otherwise.
- A demo, prediction, or visualisation that makes the result concrete.
- What you'd try next with more time.

Negative results are welcome — a clear explanation of what didn't work and why is often more useful than a marginal win.

The Challenge: Fine-tuning OpenFold3 on PDE10A

As a reference point and worked example, we are sharing the setup and results of a recent internal experiment. You can reuse this target and dataset directly, or treat it as a familiar benchmark while exploring your own approach.

Goal

Assess whether fine-tuning the public OpenFold3 model on a very small set of protein–ligand structures can meaningfully improve prediction quality for a specific drug discovery context, without degrading general performance. The target is human phosphodiesterase 10A (PDE10A) — a class where pose accuracy and SAR alignment are critical for design decisions.

The practical question:

Can we correct systematic pose errors of OpenFold3 for a specific target and chemotype using only a handful of protein–ligand complexes?

This short experiment aims to:

1. Show that small numbers of liganded structures can materially increase the capabilities of co-folding models.
2. Provide enough detail that a structural modelling team could reproduce or extend the experiment in their own stack.

Setup

We evaluated OpenFold3 fine-tuning on a subset of 27 structures from the PDE10A dataset published by Roche in 2022 (<https://link.springer.com/article/10.1007/s10822-022-00478-x>). It met two key requirements:

- Held out from the OF3 training set (published after the 2021 training cutoff).
- The base OF3 model was performing poorly on these structures.

Treated as a realistic low-n fine-tuning scenario:

- **Target:** human PDE10A
- **Model:** OpenFold3 (public weights, late 2025)
- **Hardware:** single NVIDIA H100 GPU
- **Total wall-clock for fine-tuning:** ~20 hours

Data: Training and Evaluation Splits

Training set (10 complexes — used for fine-tuning):

5SDY, 5SIQ, 5SI7, 5SIG, 5SI5, 5SI8, 5SIY, 5SG5, 5SGL, 5SIH

Evaluation set (17 held-out complexes — no gradient updates):

5SH0, 5SE0, 5SHR, 5SJL, 5SH8, 5SF4, 5SFG, 5SE5, 5SHK, 5SEE, 5SFL, 5SJU, 5SKE, 5SKU, 5SKO, 5SEA, 5SKR

The dataset has been uploaded to `/apheris/apherisfold_inputs.zip` on your machine.

Reference Fine-tuning Protocol

A minimal, single-target fine-tuning setup. Useful as a starting point — feel free to deviate.

- Starting weights: public OpenFold3
- Objective: standard OpenFold3 structure loss on protein–ligand complexes
- Steps: ~350 gradient steps over the 10 training complexes
- EMA decay: 0.99 (decreased from default 0.999) to allow meaningful model change in a small number of gradient steps
- Learning rate: 0.0003 (decreased from 0.0018)
- Warmup steps: 50 (decreased from 1000)
- MSAs / templates: as in baseline OpenFold3, but without templates

All training was run via ApherisFold in a private environment, with the PDE10A complexes staying inside the organisation's own infrastructure.

Accessing Your Team's Machine

Each team is provisioned with a dedicated VM and a private SSH key. Your team lead will receive the IP address and your private key via a BitWarden Send link.

Throughout the steps below, replace `<team-key>` with the filename of the key you downloaded (e.g. `team-3`) and `<vm-ip>` with the IP address you were given.

SSH user:

lyceum

1. Install your private key

Download the key from the BitWarden Send link, then move it into your ~/.ssh directory and lock down the permissions:

```
mv ~/Downloads/<team-key> ~/.ssh/<team-key>
chmod 600 ~/.ssh/<team-key>
```

2. Tunnel to the Hub UI

Open an SSH tunnel that forwards the Hub UI to your local machine:

```
ssh -i ~/.ssh/<team-key> -N -L 8081:localhost:8080 lyceum@<vm-ip>
```

Then navigate to <http://localhost:8081> in your browser.

3. Direct SSH access

To get a shell on the VM:

```
ssh -i ~/.ssh/<team-key> lyceum@<vm-ip>
```

Fine-tuning with ApherisFold

ApherisFold lets you fine-tune co-folding models on proprietary structures through a simple UI — without your data leaving the IT environment. This is currently a beta feature; future releases will support scaling to larger GPU clusters, fine-tuning on affinity data, and more powerful analysis tools.

Creating a job

A step-by-step visual walkthrough of creating a fine-tuning job is included at the end of this guide. The summary below covers the same flow:

Hyperparameter reference

Parameter	What it does
maximum_training_time	How long (in seconds) the fine-tuning job should run.
batch_size	Number of cropped structure samples aggregated before each weight update. Independent of training-set size. Smaller values produce noisier gradients (helps with overfitting, can limit convergence) and may need a smaller learning rate for stability.
learning_rate	Higher values train faster but risk instability or worse convergence.
warmup_steps	How quickly the learning rate is scaled up at the start of training. Larger = longer but potentially more stable.
num_gradient_steps_per_epoch	How often metrics and checkpoints are written. Smaller = higher-resolution curves but slower training (more validation passes) and faster disk fill.
training_seed	Model seed. Tweak to understand fine-tuning variance — you may end up with quite different models.
data_seed	Dataloader seed. Generally lower impact than training_seed; both must match for full reproducibility.

precision	May need to be modified for GPUs that do not support bf16.
crop_size	Tokens (residues / nucleic acids / ions / small-molecule atoms) sampled stochastically per training forward pass. Larger = more GPU memory; smaller may struggle with global conformational states but acts as regularisation. Generally set below the combined sequence length of protein chains in each datapoint. No cropping on validation — a 40 GB GPU supports validation structures up to ~1700 tokens.
ema_decay	Rate at which fine-tuned weights lose their bias toward the base model. Roughly $\text{ema_decay}^{(\text{num_gradient_steps_per_epoch} \times \text{num_epochs})}$ of the base weights remain. Important for controlling overfitting on small datasets — increase to retain more generalisation.

Suggested starting hyperparameters (PDE10A reference run)

If you're fine-tuning on PDE10A or a comparable low-n protein–ligand setup, the following values reproduce the reference experiment:

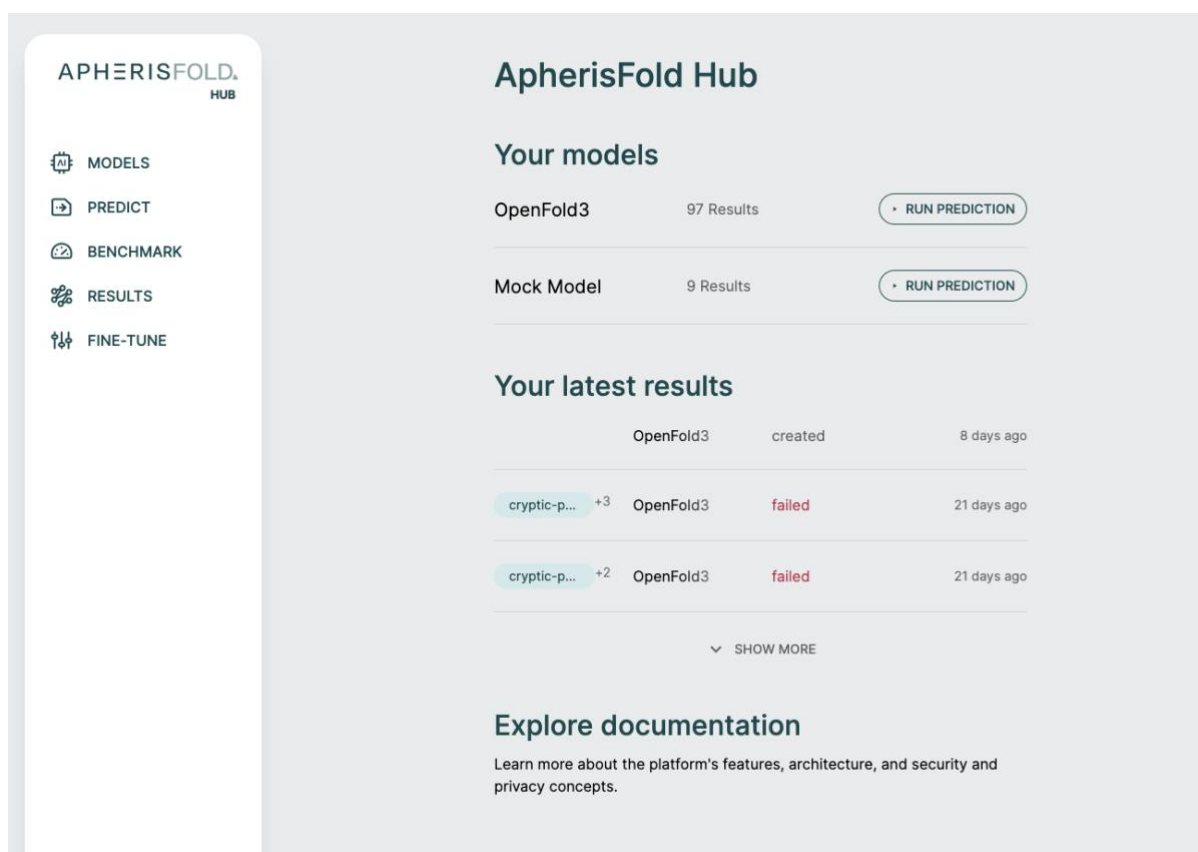
- learning_rate: 0.0003
- warmup_steps: 50
- ema_decay: 0.99
- Total gradient steps: ~350
- Templates: disabled; MSAs as in baseline OF3

Visual walkthrough: creating a fine-tuning job

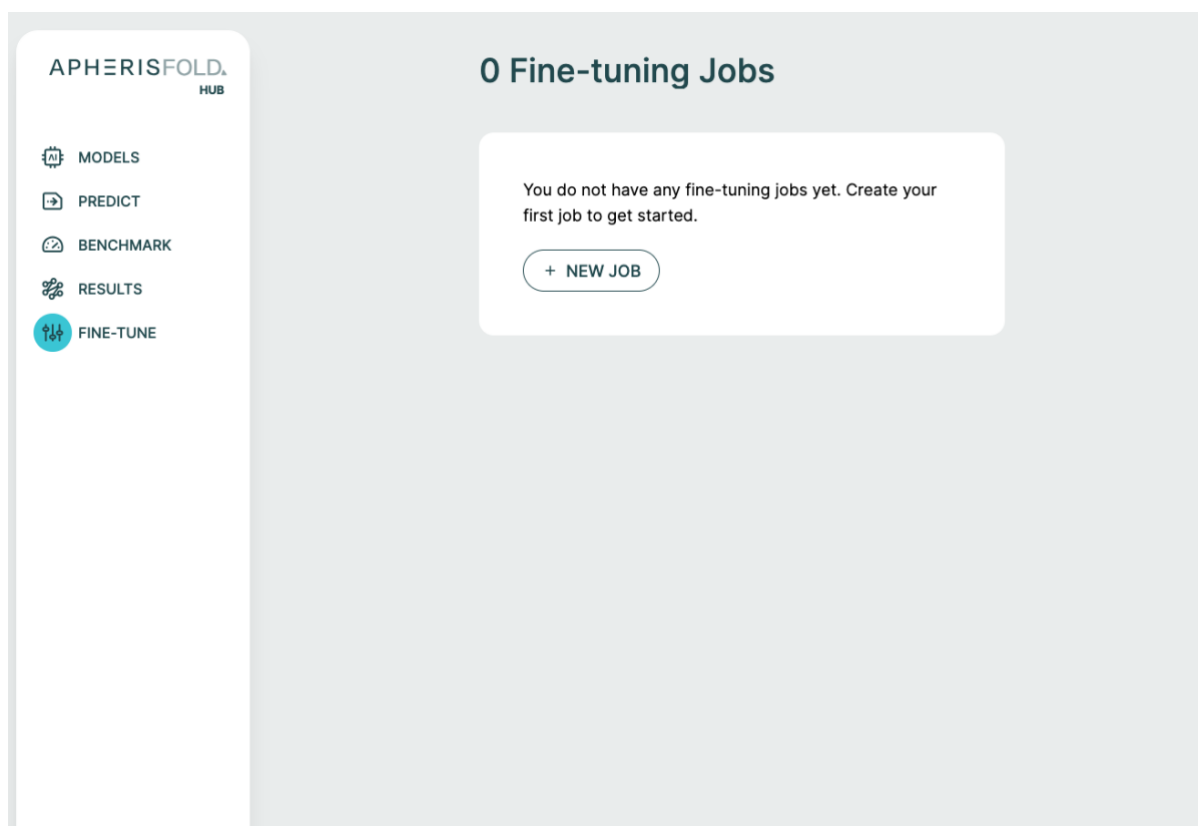
You can find the full docs on fine-tuning here: <https://www.apheris.com/docs/hub/fine-tuning.html>

Below are the screens you will see when setting up a job, with notes on each step.

1. Load ApherisFold



2. Go to fine-tuning



3. Create a new job and enter the job name

APHERISFOLD.
HUB

MODELS

PREDICT

BENCHMARK

RESULTS

FINE-TUNE

< New Fine-tuning Job

Model and Weights

openfold3 (4.0.0)

Job Name

Description (Optional)

Files

Settings

Training Files

Validation Files

START FINE-TUNING

CANCEL

- Drag and drop the training data (cifs and a3ms) into the “Training Files” box. Since all ligands are in the CCD, you don’t need to provide custom ligand definitions (SDF).

APHERISFOLD.
HUB

MODELS

PREDICT

BENCHMARK

RESULTS

FINE-TUNE

< New Fine-tuning Job

Model and Weights

openfold3 (4.0.0)

Job Name

My job

Description (Optional)

Files

Settings

Training Files

Validation Files

START FINE-TUNING

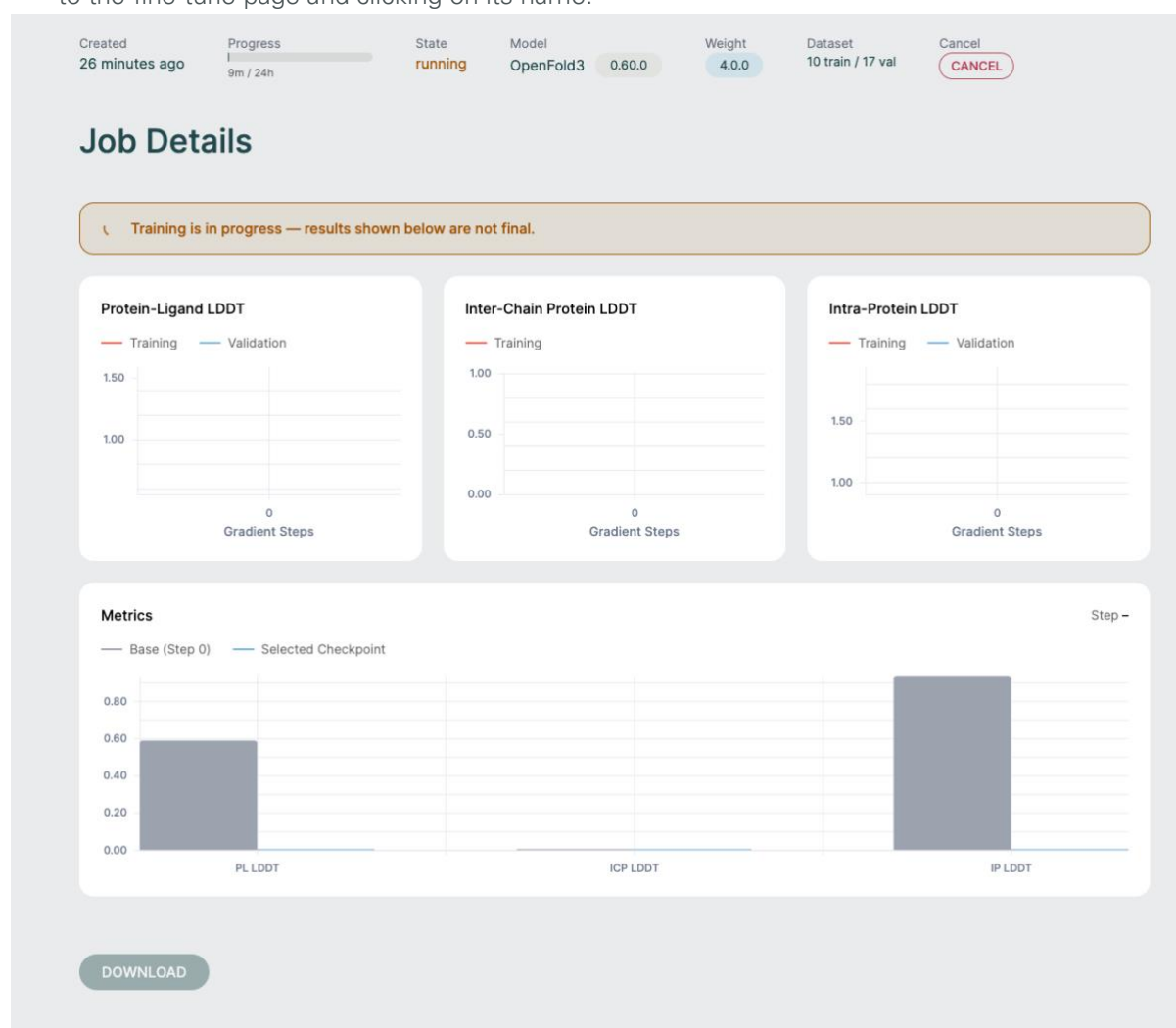
CANCEL

File Name	Size	MSA	SDF	Status
5sdy.cif	348.6 kB	⌵	-	⌵
5sg5.cif	1.1 MB	⌵	-	⌵
5sgl.cif	1.2 MB	⌵	-	⌵
5si5.cif	1.1 MB	⌵	-	⌵
5si7.cif	1.2 MB	⌵	-	⌵
5si8.cif	1.2 MB	⌵	-	⌵
5sig.cif	1.2 MB	⌵	-	⌵
5sih.cif	1.2 MB	⌵	-	⌵
5siq.cif	1.2 MB	⌵	-	⌵
5siy.cif	1.2 MB	⌵	-	⌵

- Wait a couple of mins while everything uploads. You will see MSAs listed as missing at first, but over time they should all turn green.

- You can modify your config in the "Settings" pane

7. Once you're ready, hit "Start fine-tuning"! You can view the status of your job by going back to the fine-tune page and clicking on its name:



Deploying Your Custom Weights

Once your fine-tuning job has finished, you can serve the new weights from your team's ApherisFold deployment. The flow is: download the checkpoint, drop it into the weights directory on the VM, register it in the config, and redeploy.

Full guide here: <https://www.apheris.com/docs/hub/apherisfold-application.html#customizing-model-weights>

1. Download the checkpoint

On the fine-tuning job's results page in ApherisFold, select a checkpoint and download it. For OpenFold3 you will get a single file (if your checkpoint is named something different, please rename it to this):

```
of3_ft3_v1.pt
```

2. Place the file on the VM

SSH into your VM (see Accessing Your Team's Machine above) and copy the checkpoint into a new sub-directory of the weights mount. The expected layout is:

```
weights_mount/
```



```
|—— additional_weights.json
|—— fine-tuned/
|   |—— of3_ft3_v1.pt
```

3. Register the weights in additional_weights.json

Edit additional_weights.json so it lists your new weight set. A minimal entry:

```
{
  "available_weights": [
    {
      "model_type": "openfold3",
      "version": "3.0.0-fine-tuned",
      "description": "PDE10A fine-tuned weights",
      "model_scope": ["inference"],
      "mounted_path": "/weights/openfold3/fine-tuned"
    }
  ]
}
```

Use a unique version string per weight set, and make sure mounted_path matches the directory you created in step 2 (under the container's /weights mount).

4. Redeploy

From the deployment directory on the VM, run:

```
./deploy_apherisfold
```

To confirm everything mounted correctly:

```
./deploy_apherisfold diagnose
```

5. Use the weights

Reload the Hub UI and head to the Predict page. Your custom version (e.g. 3.0.0-fine-tuned) will appear in the weight-version dropdown next to the default weights.

Good luck!

If you hit anything blocking — VM access, ApherisFold UI, dataset prep — flag it in the team channel and we'll jump in.